Examples of Shaping the Output Patterns

Conversation Log | June 27, 2026

USER

create examples for the Shaping the output pattern category

CLAUDE

Here's one example for each of the four patterns under Shaping the Output — Few-Shot, Role/Persona Framing, Template Filling, and Constraint-Based Prompting.

Few-Shot / Worked Examples

Show 1-3 examples of the exact input→output mapping you want before the real request, instead of describing the format in the abstract — letting the model pattern-match to a concrete shape rather than interpret a description.

> *"I want function doc comments in this exact style for TextFinder1's core crate. Here are two examples:"*

```
/// Parses a comma-separated extension list into normalized form.
/// Returns Err if the input is empty or contains only whitespace.
pub fn parse_extensions(input: &str) -> Result<Vec<String>, MatchError>

/// Checks whether a line contains the configured pattern.
/// Empty lines never match, regardless of pattern.
pub fn matches(&self, line: &str) -> bool
```

> *"Now write doc comments in this same style for find_matches and from_args."*

The model mirrors the structure exactly — a one-line summary, a second line stating a specific edge-case behavior — rather than guessing at a generic Rustdoc style that might use # Arguments sections or a different tone.

Why it matters: a description like "write good doc comments" leaves "good" open to interpretation; two real examples remove that ambiguity entirely.

Role / Persona Framing

Assign an expert persona to bias the model's vocabulary, rigor, and priorities — useful when you want a specific lens applied, not just "an answer."

> *"Review this TextFinder1 directory-traversal function as a security auditor would — focus specifically on path handling, symlink behavior, and anything that could let traversal escape the intended root directory."*

Framed this way, the model prioritizes differently than a generic code review would: it checks whether .. segments in user-supplied paths are canonicalized before comparison, whether symlinks pointing outside the search root are followed silently, and whether error messages leak full filesystem paths — concerns a general "does this look clean" review might skip past in favor of style or naming.

Why it's distinct from Few-Shot: Few-Shot fixes the output format with concrete examples; Role Framing fixes the evaluative lens the model reasons through, without specifying what the output should literally look like.

Template Filling

Give a fixed output structure with named sections, and have the model populate it rather than free-form generate — this is the backbone of your constitution/spec/plan/tasks setup.

> *"Fill in this exact template for the new --regex feature spec — don't add sections, don't remove any, leave anything you're unsure about as <!-- open question --> rather than guessing:"*

```
## Goal
## Behavior
## Edge Cases
## Out of Scope
## Acceptance Criteria
```

The model populates each section in order, and where it genuinely doesn't know your intent — say, whether --regex should support named capture groups — it leaves an explicit open question instead of silently picking an answer.

Why it's distinct from Few-Shot: Few-Shot shows what a filled example looks like; Template Filling gives the empty skeleton itself and constrains the model to that exact structure, with no freedom to reorganize or omit sections.

Constraint-Based Prompting

Front-load hard constraints — version limits, forbidden APIs, style rules — so the model self-filters during generation, rather than producing something you then have to catch and correct.

> *"Write the new matcher logic for TextFinder1's core crate. Hard constraints: must compile on Rust 1.70 (no newer edition features), no unsafe blocks anywhere, no new dependencies beyond what's already in Cargo.toml, and follow the existing Result<T, MatchError> error pattern rather than panicking."*

With these stated up front, the model avoids reaching for a newer stdlib API that needs Rust 1.75+, doesn't introduce a tempting-but-unnecessary unsafe block for a minor performance gain, and doesn't pull in a new crate for convenience — because the boundaries were given before generation started, not discovered afterward in a failed build.

Why it's distinct from Template Filling: Template Filling constrains the document structure; Constraint-Based Prompting constrains the content and implementation choices within whatever structure or format is being produced.

Together: Few-Shot and Template Filling both fix shape (one by example, one by explicit skeleton), while Role Framing and Constraint-Based Prompting both fix substance (one by lens, one by hard boundary) — so picking the right one depends on whether your problem is "the output looks wrong" or "the output reasons about the wrong things."